مدن المهن و الكفاءات
Cités des métiers et des compétences

مكتب التكوين المهني و إنعاش الشغل
Office de la Formation Professionnelle et de la Promotion du Travail



M205
Développer en Backend
Rappel des fondamentaux

Mme Imane FRITET

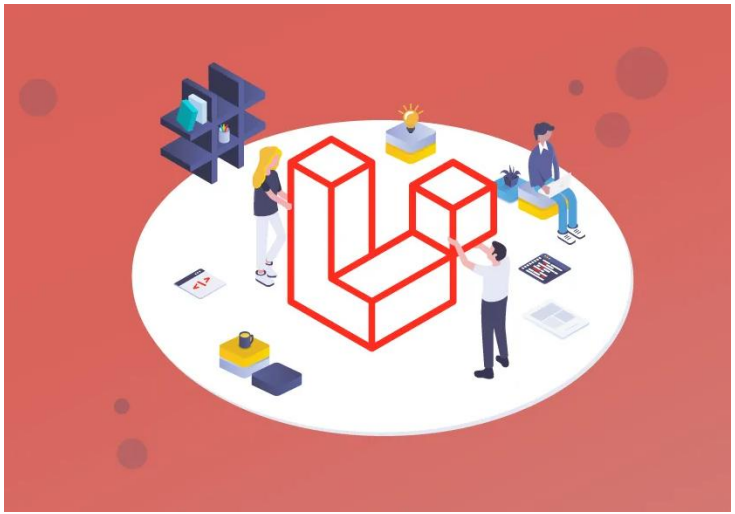
imane.fritet@ofppt-edu.ma

Secteur : Digital et IA

Prérequis

- ✓ Langage PHP
- ✓ POO(encapsulation ,surcharge ,héritage …)
- ✓ PDO(PDO, PDOStatement, PDOException)
- ✓ MVC(Model, View Controller)
- ✓ Serveurs(service, Port ,Protocol …)
- ✓ Base de Donnée(SQL)

RAPPEL
Programmer en PHP



1. Rappels des fondamentaux des sites web dynamiques
- Notions de bases
- Programmation orienté objet
- Manipuler des données dans des bases MySQL avec PDO
2. Présentation du MVC (Modèle - Vue - Contrôleur)
- Créer différentes couches du modèle MVC
- Création des composants Contrôleur
- Création des Vues
- Création des composants Modèle

Mme Imane FRITET imane.fritet@ofppt-edu.ma Secteur : Digital et IA

RAPPEL

Concepts de base de la POO en PHP : Définition de classe

- Les **objets** possèdent des propriétés et des comportements propres.
- Une même famille d'objets, c'est-à-dire avec des caractéristiques communes, peut être regroupée dans un ensemble appelé classe.
- Une **classe** permet de regrouper au sein d'une même structure le format des données qui définissent l'objet et les fonctions destinées à manipuler ces objets.

```php
<?php

class Voiture
{
    //Attributs
    private $couleur;
    private $kilometrage;

    //Méthodes
    public function getKilometrage()
    {
        return $this->kilometrage;
    }

    public function setCouleur($couleur)
    {
        $this->couleur = $couleur;
    }

    public function rouler($distance)
    {
        $this->kilometrage += $distance;
    }
}
```

Concepts de base de la POO en PHP : Définition d'un objet

- Un **objet** : un élément particulier d'une classe, est appelé instance d'une classe.
- La déclaration d'un objet revient à instancier une classe avec **new.**

Résultat:

Kilométrage : 10

renault est un objet de la classe voiture

```php
<?php

//Déclaration d'une instance
$renault = new voiture();

//Utilisation
$renault->rouler(10);
echo "Kilométrage : " . $renault->getKilometrage() . "<br>";

//Vérification
if (is_a($renault, "voiture"))
{
    echo "renault est un objet de la classe voiture";
}
```

RAPPEL

RAPPEL

Concepts de base de la POO en PHP : L’encapsulation et visibilité

L'encapsulation est un principe fondamental de la POO. Elle consiste à protéger les données d'une classe en restreignant l'accès direct à ses propriétés (attributs) et en les rendant accessibles uniquement par des méthodes spécifiques (**getters et setters**).

La visibilité est un concept de la programmation orientée objet (POO) qui détermine comment les propriétés (attributs) et méthodes (fonctions) d'une classe peuvent être accessibles depuis l'intérieur et l'extérieur de cette classe. PHP utilise trois niveaux de visibilité : **public**, **protected**, et **private**.

Concepts de base de la POO en PHP : Membres statiques

RAPPEL

En PHP il est possible de créer des propriétés de classe. Leur valeur est la même pour toutes les instances d'une même classe.

- Les propriétés statiques sont déclarées avec **static** et doivent être accédées via la syntaxe **NomDeClasse::$nomDePropriété**.
- Les méthodes statiques sont également déclarées avec **static** et peuvent être appelées via la syntaxe **NomDeClasse::nomDeMéthode()**.

```php
<?php

class Bourse
{
    //attribut statique
    public static $lieu = "Rabat";

    //méthode statique
    public static function infosGenerales()
    {
        $heure = date("H:i:s");
        $texte = self::$lieu . ", il est " . $heure;
        return $texte;
    }
}
echo Bourse::$lieu ."<br>"; // Rabat
echo Bourse::infosGenerales(); // Rabat, il est 10:30:25
```

RAPPEL

Concepts de base de la POO en PHP : Constructeur

Quand une instance d'une classe d'objet est créée au moment de l'instanciation d'une variable avec **new**, une fonction particulière est exécutée. Cette fonction s'appelle le constructeur. Elle permet d'initialiser chaque instance pour que ses propriétés aient un contenu cohérent.

En PHP, cette méthode est spéciale est appelé **__construct**, et est appelée automatiquement au moment de la création d'un objet.

```php
<?php

class Counter
{
    public static $count = 0;

    public function __construct() {
        self::$count++;
    }
}
```

RAPPEL

Concepts de base de la POO en PHP : Constructeur

```php
//Définition de la classe
class voiture
{    //Attributs
    private string $couleur;
    private int $kilometrage;
    //constructeur
    public function __construct(string $couleur,
int $kilometrage)
    {
        $this->couleur = $couleur;
        $this->kilometrage = $kilometrage;
    }
public function afficher()
    {
        return "Voiture => Couleur : " . $this-
>couleur . ", Kilométrage : " . $this->kilometrage;
    }
}
```

```php
//Utilisation:

$v1 = new voiture("Rouge", 1000);
echo $v1->afficher();
```

Résultat :

Voiture => Couleur : Rouge, Kilométrage : 1000

```php
$v2 = new voiture();
echo $v2->afficher();
```

Résultat :

Fatal error: Uncaught ArgumentCountError: Too few arguments to function voiture::__construct(), ….

Concepts de base de la POO en PHP : Héritage

L'héritage consiste en la création d'une nouvelle classe dite classe dérivée à partir d'une classe existante dite classe de base ou classe parente.

L'héritage permet de :

- Récupérer le comportement standard d'une classe d'objet (classe parente) à partir de propriétés et des méthodes définies dans celles-ci,
- Ajouter des fonctionnalités supplémentaires en créant de nouvelles propriétés et méthodes dans la classe dérivée,
- Modifier le comportement standard de la classe parente en surchargeant certaines méthodes de la classe parente dans la classe dérivée.

RAPPEL

Concepts de base de la POO en PHP : Héritage

```php
class animal
{
    protected $poids;
    protected $age;

    public function respirer()
    {
        echo "Je respire!!<br>";
    }
}
```

```php
class chien extends animal
{
    public function aboyer()
    {
        echo "Wooaf!!<br>";
    }
}
```

```php
$a = new animal();
$a->respirer();
```

Résultat :

Je respire!!

```php
$c = new chien();
$c->respirer();
$c->aboyer();
```

Résultat :

Je respire!!
Wooaf!!

RAPPEL

Manipuler des données dans les bases MySQL avec PDO : Introduction

L'extension **PHP Data Objects (PDO)** définit une excellente interface pour accéder à une base de données depuis PHP.

Chaque pilote de base de données implémenté dans l'interface PDO peut utiliser des fonctionnalités spécifiques de chacune des bases de données en utilisant des extensions de fonctions.

3 classes principales:

PDO : une instance de PDO représente la connexion à une base de données.

⇒ le plus souvent une seule instance de PDO par exécution de PHP

PDOStatement : une instance de PDOStatement représente une requête vers la base.

⇒ permet de préparer la requête puis de consulter son résultat.

PDOException :Représente une erreur émise par PDO

RAPPEL

RAPPEL

Connexion à une base de données MYSQL avec PDO

```php
<?php

//Data Source Name :
//Driver:mysql
//Serveur d'hébérgement: host=localhost
//Nom de la base de données: dbname=test_db
$dsn = "mysql:host=localhost;dbname=test_db";
//Nom de l'utilisateur pour se connecter au serveur mysql:
$user = "root";
//Mot de passe:
$pass = "";
try {
    $db = new PDO($dsn, $user, $pass);
    echo "Connexion réussie";
} catch (PDOException $e) {
    //die :affiche le message et quitte le script PHP actuel.
    die('Erreur : ' . $e->getMessage());
}
```

PDO: Requêtes préparées

PDO::prepare

méthode : **PDOStatement prepare($requeteSQL);**

- Prépare la requête mais ne l'exécute pas
- Bien adaptée pour les requêtes exécutées plusieurs fois.
- Mais aussi pour sécuriser les requêtes avec paramètres

l'objet **PDOStatement** permettra (ensuite) de lancer l'exécution.

```
$stmt = $connexion->prepare( "select nom,prenom from tab");
$stmt->execute();
```

Une fois la requêtes exécutée, l'objet PDOStatement permet d'explorer le résultat de la requête

RAPPEL

PDO: Requêtes préparées paramétrées

PDO::prepare

- La requête peut comporter des "paramètres"...
- …dont on fixe la valeur au moment de l'exécution
- Sécurité contre l'injection de code

```
$stmt = $connexion->prepare("select nom, prenom from tab where nom=:name");

$stmt->execute(array(":name"=>"Ahmed"));

$stmt->execute(array(":name"=>"Hicham"));
```

Voir aussi la méthode bindValue

RAPPEL

PDO : exploration des résultats

PDOStatement::fetch mixed fetch();

- Lecture ligne par ligne du résultat
- Par défaut renvoie un tableau PHP représentant une ligne.
- Renvoie false si plus aucune ligne n'est disponible

```
// pour recevoir des tableaux associatifs :
$stmt->setFetchMode(PDO::FETCH_ASSOC);
// parcours des lignes du résultat
while ($ligne = $stmt->fetch()) {
    print_r($ligne);
}
```

RAPPEL

RAPPEL

PDO : exploration des résultats

PDOStatement::setFetchMode

PDO::FETCH_ASSOC: retourne un tableau indexé par le nom de la colonne comme retourné dans le jeu de résultats

PDO::FETCH_NUM : tableau indexé par des entiers

PDO::FETCH_BOTH (défaut): retourne un tableau indexé par les noms de colonnes et aussi par les numéros de colonnes, commençant à l'index 0, comme retournés dans le jeu de résultats

PDO::FETCH_CLASS : instance d'une classe à fournir en 2ème argument

PDO::FETCH_OBJ : retourne un objet anonyme avec les noms de propriétés qui correspondent aux noms des colonnes retournés dans le jeu de résultats

PDO : Exemple FETCH_OBJ

```php
//Intégrer le fichier de connexion:
require("db.php");
$sel = $db->prepare("SELECT * FROM utilisateur");
            // Le résultat est un tableau des dont les propriétés sont les noms de colonnes : exemple [1]->ObjetUtilisateur
$sel->setFetchMode(PDO::FETCH_OBJ);
$sel->execute();
// utlisation fetchAll
echo "<br>----------------fetchAll------------------<br>";
$TabOUtilisateur = $sel->fetchAll();
//pour voir la structure du table
echo '<pre>';
print_r($TabOUtilisateur);
echo '</pre>';
```

=> Résultat :

```
----------------fetchAll------------------

Array
(
    [0] => stdClass Object
        (
            [Id] => 1
            [Nom] => Alaoui Sara
            [Email] => alaoui.sara@gmail.com
        )

    [1] => stdClass Object
        (
            [Id] => 2
            [Nom] => Hicham
            [Email] => hicham@yahoo.fr
        )

)
```

RAPPEL

PDO : Exemple FETCH_ASSOC

```php
//Intégrer le fichier de connexion:
require("db.php");
$sel = $db->prepare("SELECT * FROM utilisateur");
            // Le résultat est un tableau des tableaux associatifs dont les clés sont les noms de colonnes : exemple [1]['nom']
$sel->setFetchMode(PDO::FETCH_ASSOC);
$sel->execute();
// utlisation fetchAll
echo "<br>----------------fetchAll-------------------<br>";
$TabUtilisateur = $sel->fetchAll();
//pour voir la structure du table
echo '<pre>';
print_r($TabUtilisateur);
echo '</pre>';
```

=> Résultat :

```
----------------fetchAll-------------------

Array
(
    [0] => Array
        (
            [Id] => 1
            [Nom] => Alaoui Sara
            [Email] => alaoui.sara@gmail.com
        )

    [1] => Array
        (
            [Id] => 2
            [Nom] => Hicham
            [Email] => hicham@yahoo.fr
        )

)
```

RAPPEL

RAPPEL

PDO : Exemple FETCH_NUM

```php
//Intégrer le fichier de connexion:
require("db.php");
$sel = $db->prepare("SELECT * FROM utilisateur");
            // Le résultat est un tableau des tableaux dont
les colonnes sont référencés par des indices commençant par 0 :
$sel->setFetchMode(PDO::FETCH_NUM);
$sel->execute();
// utlisation fetchAll
echo "<br>----------------fetchAll------------------<br>";
$TabUtilisateur = $sel->fetchAll();
//pour voir la structure du table
echo '<pre>';
print_r($TabUtilisateur);
echo '</pre>';
```

=> Résultat :

```
----------------fetchAll------------------

Array
(
    [0] => Array
        (
            [0] => 1
            [1] => Alaoui Sara
            [2] => alaoui.sara@gmail.com
        )

    [1] => Array
        (
            [0] => 2
            [1] => Hicham
            [2] => hicham@yahoo.fr
        )

)
```

PDO : Exemple FETCH_BOTH

```php
//Intégrer le fichier de connexion:
require("db.php");
$sel = $db->prepare("SELECT * FROM utilisateur");
          // Le résultat associant le mode
FETCH_ASSOC et FETCH_NUM
$sel->setFetchMode(PDO::FETCH_BOTH);
$sel->execute();
// utlisation fetchAll
echo "<br>---------fetchAll--------<br>";
$TabUtilisateur = $sel->fetchAll();
//pour voir la structure du table
echo '<pre>';
print_r($TabUtilisateur);
echo '</pre>’;
```

=> Résultat :

```
----------fetchAll----------

Array
(
    [0] => Array
        (
            [Id] => 1
            [0] => 1
            [Nom] => Alaoui Sara
            [1] => Alaoui Sara
            [Email] => alaoui.sara@gmail.com
            [2] => alaoui.sara@gmail.com
        )

    [1] => Array
        (
            [Id] => 2
            [0] => 2
            [Nom] => Hicham
            [1] => Hicham
            [Email] => hicham@yahoo.fr
            [2] => hicham@yahoo.fr
        )

)
```

RAPPEL

PDO : Exemple FETCH_CLASS

```php
//Intégrer le fichier de connexion:
require("db.php");
require("utilisateur.class.php");


$sel = $db->prepare("SELECT * FROM utilisateur");
            // Le résultat est un tableau des objets de type 'utilisateur' dont les propriétés sont les noms de colonnes : exemple [1]->ObjetUtilisateur
$sel->setFetchMode(PDO::FETCH_CLASS,  utilisateur::class);
$sel->execute();
// utlisation fetchAll
echo "<br>----------------fetchAll------------------<br>";
$TabOUtilisateur = $sel->fetchAll();
//pour voir la structure du table
echo '<pre>';
print_r($TabOUtilisateur);
echo '</pre>';
```

utilisateur.class.php > ...

```php
<?php
class utilisateur
{
    private int $id;
    private string $nom;
    private string $email;
}
```

=> Résultat

```
----------fetchAll----------
Array
(
    [0] => utilisateur Object
        (
            [Id] => 1
            [Nom] => Alaoui Sara
            [Email] => alaoui.sara@gmail.com
        )

    [1] => utilisateur Object
        (
            [Id] => 2
            [Nom] => Hicham
            [Email] => hicham@yahoo.fr
        )

)
```

RAPPEL

PDO : Exemple de récupération d'une seule ligne :

```php
require("db.php");


$sel = $db->prepare("SELECT * FROM utilisateur where id=:id");
//Passage de valeur au paramètre :id
$sel->execute([":id" => 2]);
//Une seule lecture avec fetch en précisant le mode de fetch
$ObjUtilisateur = $sel->fetch(PDO::FETCH_OBJ);
if ($ObjUtilisateur) {
    echo "Id : " . $ObjUtilisateur->Id . "<br>";
    echo "Nom : " . $ObjUtilisateur->Nom . "<br>";
    echo "Email : " . $ObjUtilisateur->Email . "<br>";
}
```

=> Résultat :

Id : 2
Nom : Hicham
Email : hicham@yahoo.fr

RAPPEL

PDO : Autres méthodes d ’exécution de requêtes

PDO::exec

méthode : **int exec($requeteSQL)**

bien adaptée pour les requêtes autres que SELECT renvoie le nombre de lignes affectées

```
$nbLignes = $connexion->exec("delete from tab");
```

PDO::query

méthode : **PDOStatement query($requeteSQL)**

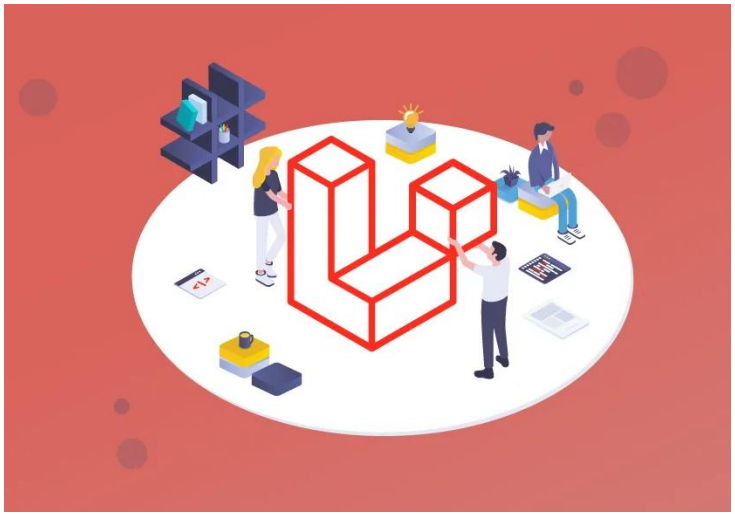
- bien adaptée pour les requêtes SELECT exécutées une fois.
- prépare et exécute une requête SQL en un seul appel de fonction, retournant la requête en tant qu'objet Postalement.

Exemple d’utilisation

```
$res = $connexion->query("select nom,prenom from tab")
```

RAPPEL

RAPPEL
Programmer en PHP



1. Rappels des fondamentaux des sites web dynamiques
- Notions de bases
- Programmation orienté objet
- Manipuler des données dans des bases MySQL avec PDO
2. Présentation du MVC (Modèle - Vue - Contrôleur)
- Créer différentes couches du modèle MVC
- Création des composants Contrôleur
- Création des Vues
- Création des composants Modèle

Introduction - Comment avoir une structure propre et pro pour mon code ?

Pour avoir un code professionnel il faut avoir un code:

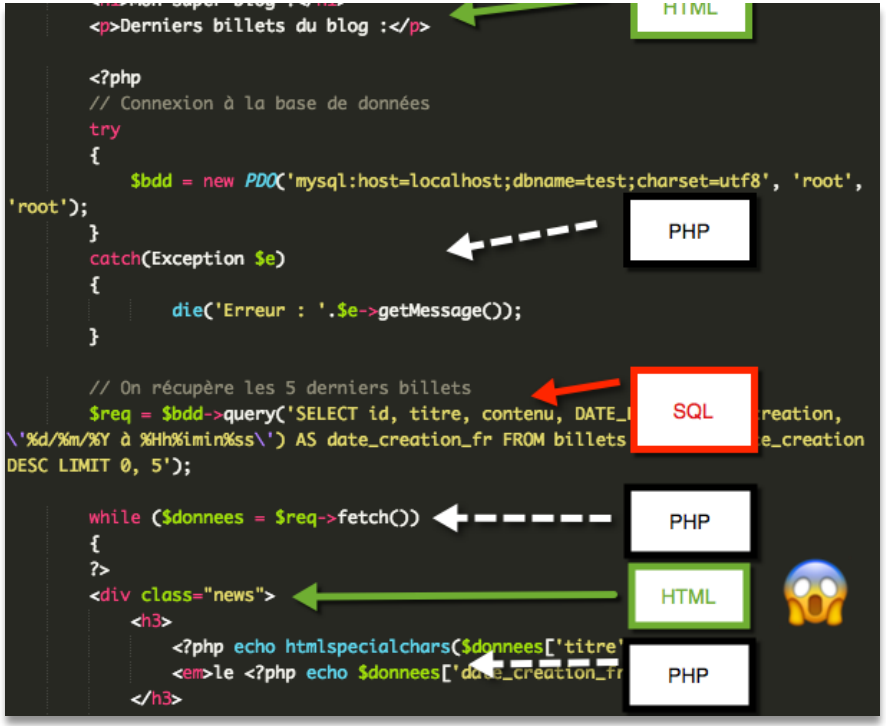
- ✓ **Modulaire** : généralement découpé en de nombreux fichiers, chaque fichier a un et un seul rôle à la fois.
- ✓ **Découplé** : les fichiers sont conçus pour fonctionner indépendamment les uns des autres.
- ✓ **Clair** : chaque fichier est généralement assez court (jamais des fichiers de 1000 lignes !).
- ✓ **Réutilisable**: si un jour nous avons codé un fichier utile, nous pouvons nous en resservir dans un autre projet, ou dans un autre endroit du même projet. On gagne du temps en n'ayant pas à tout refaire à chaque fois !
- ✓ **Évolutif**: quand quelqu'un vient vous demander une nouvelle fonctionnalité, il est facile de l'ajouter. Vous n'avez pas peur de tout casser. Vous savez que ça va marcher, et votre code ne sera pas plus compliqué.

RAPPEL

RAPPEL

Introduction - Comment avoir une structure propre et pro pour mon code ?

Tout est dans un seul fichier. On y mélange **des opérations en PHP, des requêtes SQL et du code HTML.**



Présentation de l'architecture MVC

M → **Model :** Les modèles seront les éléments qui se chargeront des échanges avec la base de données (CRUD). On ne mettra pas de traitement dans ces fichiers, uniquement des requêtes.

V → **View :** Les vues contiendront uniquement le code HTML destiné à structurer les pages.

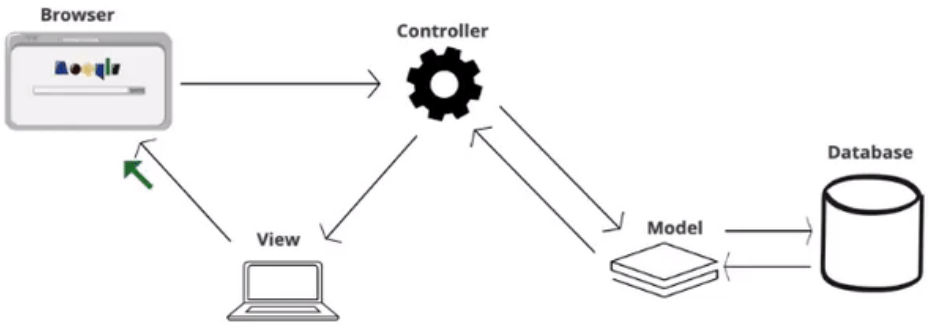
C → **Controller :** Les contrôleurs contiendront toute l'intelligence de l'application, le traitement des données en vue de leur affichage, par exemple.

RAPPEL

RAPPEL

Fonctionnement de l'architecture MVC

1. L'utilisateur effectue une action dans le navigateur (comme une recherche).
2. La requête arrive au contrôleur, qui consulte le modèle pour obtenir des données.
3. Le modèle traite la demande en interrogeant ou mettant à jour la base de données.
4. Le modèle envoie les données au contrôleur.
5. Le contrôleur transmet les données à la vue, qui les affiche dans un format compréhensible par l'utilisateur.

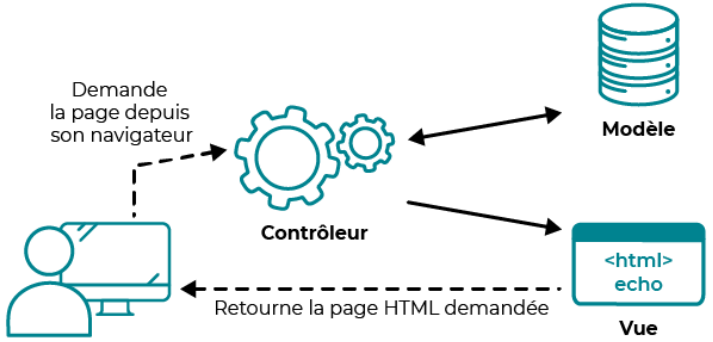


RAPPEL

Fonctionnement : Rôle de Modèle

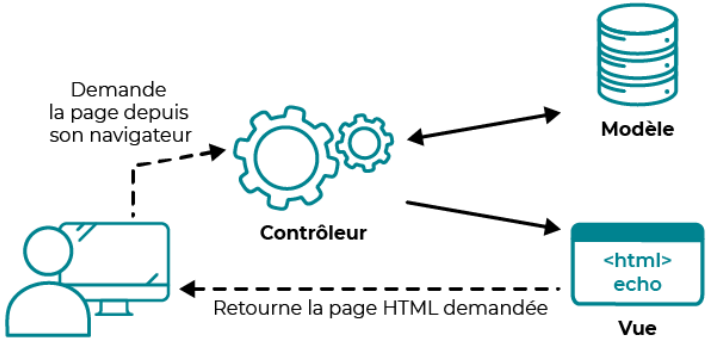
Représente la partie de l'application qui exécute la logique métier. Cela signifie qu'elle est responsable de récupérer les données et faire des traitements, validations, et beaucoup d'autres tâches concernant la manipulation des données.

- **Données de l'application**
- **Méthodes manipulant ces données**
- **Stockage et extraction de la BD**



Fonctionnement : Rôle de Vue

- Cette partie se concentre sur l'affichage.
- S'occupe des interactions avec l'utilisateur : présentation, saisie et validation des données
- Elle est responsable de retourner une réponse avec l'aide mutuelle des Model et Controller.

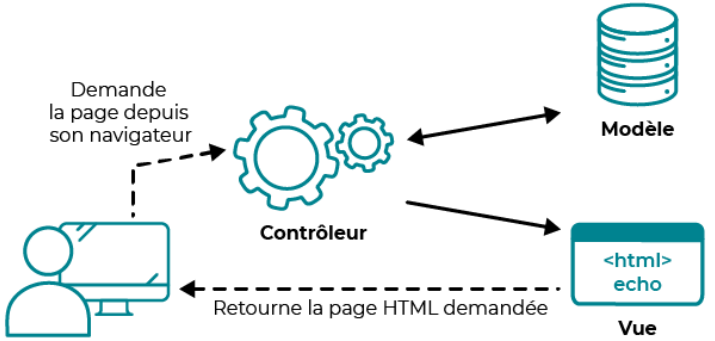


RAPPEL

RAPPEL

Fonctionnement : Rôle de Contrôleur

- Gère la dynamique de l'application, Il fait le lien entre l'utilisateur et le reste de l'application.
- Intermédiaire entre les vues et les modèles
- Accès aux données à partir du modèle
- Envoyer les données vers la partie vues



RAPPEL

Avantages & Inconvénients de l'architecture MVC

Quelques Avantages

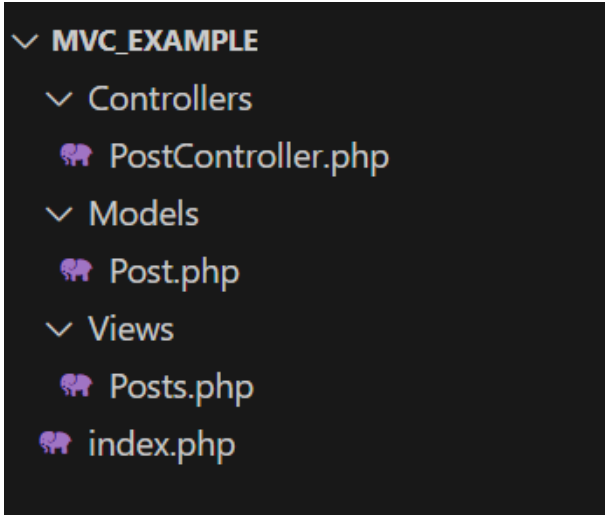
- Meilleure organisation du code.
- Possibilité de réutiliser le code.
- Meilleure gestion de temps pour la maintenance et l'évolution du site.
- Plus de facilité pour les tests unitaires.
- Conception claire et efficace grâce

Quelques Inconvénients

- Complexe pour de petites applications. Le temps accordé à l'architecture peut ne pas être rentable pour le projet.
- Augmentation de la difficulté d'implémentation.
- Le nombre important de fichiers représente une charge non négligeable dans un projet.

Exemple d'architecture MVC

- Notre code est maintenant découpé en 4 fichiers :
- Un pour le traitement PHP : il récupère les données de la base. On l'appelle le modèle. **(Ex : Models/Post.php)**
- Un pour l'affichage : il affiche les informations dans une page HTML. On l'appelle la vue. **(Ex : Views/Posts.php)**
- Un pour faire le lien entre les deux : on l'appelle le contrôleur. **(Ex : Controllers/PostController.php)**
- Un point d'entrée pour lancer l'application **(Ex : index.php)**



RAPPEL

RAPPEL

Création d'un fichier modèle :

- Dans une architecture MVC, on commence en général par écrire le modèle.
- Le modèle Post récupère les données (par exemple, en simulant une requête à une base de données).

Post.php

Models > Post.php

```php
<?php

class Post {
    public static function getAllPosts() {
        return [
            ['id' => 1, 'title' => 'First Post', 'content' => 'This is the first post.'],
            ['id' => 2, 'title' => 'Second Post', 'content' => 'This is the second post.'],
        ];
    }
}
```

RAPPEL

Création d'un fichier vue :

La vue Posts.php reçoit les données et les affiche dans un format HTML structuré.

Posts.php

Views > Posts.php

```php
<!-- views/posts.php -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Blog Posts</title>
</head>
<body>
    <h1>Blog Posts</h1>
    <?php foreach ($posts as $post): ?>
        <div>
            <h2><?php echo htmlspecialchars($post['title']); ?></h2>
            <p><?php echo htmlspecialchars($post['content']); ?></p>
        </div>
        <hr>
    <?php endforeach; ?>
</body>
</html>
```

RAPPEL

Création d'un fichier contrôleur :

- Le PostController récupère tous les articles de blog en utilisant le modèle Post.

PostController.php

Controllers > PostController.php

```php
<?php

// controllers/PostController.php
class PostController {
    public function index() {
        $posts = Post::getAllPosts();

        include 'Views/Posts.php';
    }
}
```

Création d'un fichier index :

- Le point d'entrée (index.php) initialise l'application et redirige la requête vers le contrôleur approprié.

index.php

```php
<?php

require 'Models/Post.php';
require 'Controllers/PostController.php';

$controller = new PostController();
$controller->index();
```

RAPPEL